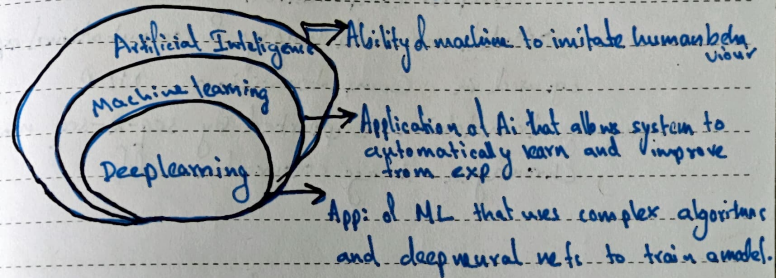


Machine learning Essentials.



Subfields of Ai:

1. Machine learning:

- Statistical techniques that gives computer the ability to learn from data
- Create stat: model to imitate ^{Intelligent} decision making
- Finding patterns in complex data to present info:
eg: deep learning, house price prediction

ML algos

↳ Supervised learning: labelled data

Unsupervised learning: unlabelled data

Reinforcement learning: learn through reward maximisation

2. Deep Learning: Uses Artificial neural networks for complex computation.
 → Models are data hungry & computational expensive.
 eg. used in Computer vision NLP
 Language translation supported by seq-2-seq model.
 Chromagen - Image colourisation model.

3. Computer Vision: Interdisciplinary field, how computers see and understand digital images and videos.

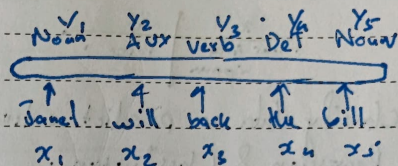
CV - [classical CV - ML
 Modern CV - Deep Learning.
 Facial recog.

4. Natural Language processing.
 Deals with human & Comp interactions

NLP - [Natural language Understanding
 Natural language ~~process~~ generation.

eg. text classification. Named entity [key pairs of data and keywords]

Part of speech tagging.
 Mapping

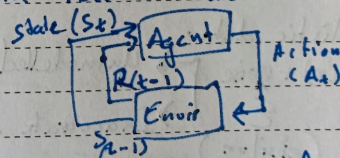


5. Automatic Speech Recognition (ASR)
 helps human to speak with computers.

speech → binary → text → Processing → Understand reply.

6. Reinforcement Learning.

- train an agent to complete a task in uncertain env
- agent receives observations and a reward from the environment & sends actions to the env
- the reward measures success rate in completing the action



eg. Autonomous robots
 self driven cars.

Supervised Learning

labelled data

x	y
Floor Area room	Price

train to classify data or predict outcomes

eg: Regression & Classification

Predict a continuous Numerical Value. Linear x, y

 $\rightarrow$ model $\rightarrow$ Price.

$$y = f(x)$$

classify or anything discrete

 $\rightarrow$ Model $\rightarrow$ %

Data:

training data set
L with labels

test data set
L without labels

Model should bring opp for test data before putting out.

in realistic data

$$y = f(x) + \text{noise}$$

ϵ
(epsilon)

Unmodelled features

Inaccurate measurements

Training Example

x, y

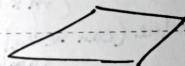
Learning algorithm $\rightarrow$ Model

x_{test}

$f(\text{Hypothesis function})$

y_{pred}

Unsupervised Learning



Find underlying ^{structure} dataset of the data-set
Structure $\rightarrow$ group unlabelled data.

$\rightarrow$ Clustering data into groups by similarity & ^(CNN) reducing dimensionality to compress the data while maintaining its structure & usefulness.
google photos: clubbing similar faces.

clusters: aim to create groups with similar features in one cluster & pts in diff clusters

Dimensionality Reduction:

reduce complexity without losing features.

Techniques:

Principal Component Analysis
Autoencoder, T-SNE, Singular value decomposition.

are dissimilar
eg. Techniques

K-Means Clustering

Hierarchical Clustering

HQ

Linear Regression

regression predicts continuous value

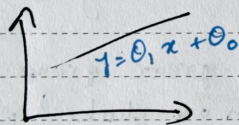
goal of algorithm is to learn a linear model to predict y for minimal x to produce o/p with low error.

(from)

Time spent (x)	Assign	Marks (y)
5	3	16
3	4	9.8
4	6	13.5
11	2	34

$n \rightarrow$ no: of features
 $m \rightarrow$ no: of training eqs.
 $x \rightarrow$ input data matrix of shape $(m \times n)$
 $y \rightarrow$ target value (can be real numbers)
 $x(i), y(i) \Rightarrow$ ith training eq
 θ $\rightarrow$ weights (parameters)
 $\hat{y}$ $\rightarrow$ hypothesis (i.e. $\rightarrow$ real no.)

(min) x (y)



step 1:

hypothesis: assume one.

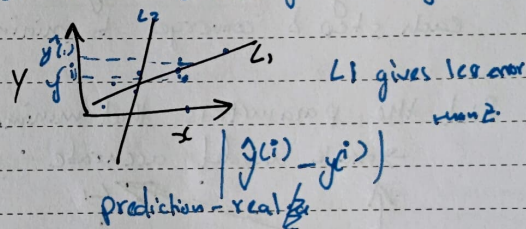
eg. y as a linear func. of x .
 hypothesis is function.

$x_{test} \rightarrow \boxed{\theta_1 x + \theta_0} \rightarrow y_{pred.}$

step 2: to Decide loss/error Function.

Given a training set, best values for the parameters

① we define an error func. that measures for each value of θ how close the prediction to actual y value. The loss func. is a meas. of how close to real & how good the algo is



$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

Better θ

↓
lower loss

↓
Better at Predictions.

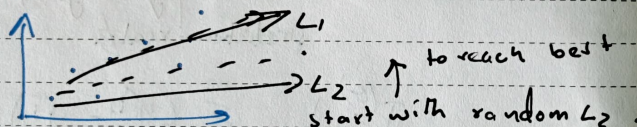
$$\hat{y} = h_{\theta}(x)$$

Step 3:

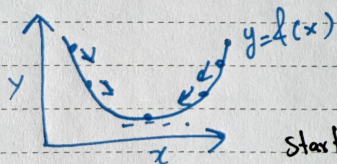
Training - an algorithm that reduces error on training data.

Start with random θ , we need an algo that iteratively improves θ by reducing $J(\theta)$ in each step & converges to minimum error

find the parameters that minimize loss, make model accurate.



Gradient Descent.



find where slope is minimum

start from random pt.

$$x = \text{rand}()$$

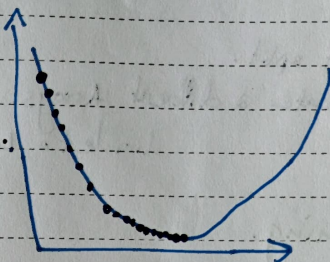
$$\text{loop } x = x - \eta \left(\frac{df}{dx} \right)$$

learning rate η (constant) $\frac{df}{dx}$ gradient

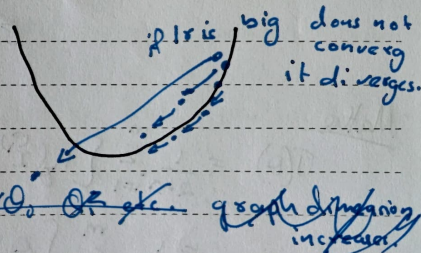
from code.

initialized a start for x anywhere.

code
A funct created.
plotted it for y .



gradient descent loop runs and scatter plotted it.
above is the o/p graph. the code/algorithm finds the descent!



when θ increases θ_0, θ_1 etc. graph diverging increases

for different parameters diff. θ_0, θ_1 [features]

Training loop.

Bring all features to similar scale

So technique Standardization. values centred around mean.
with a unit std deviation.

$$x = \frac{x - x.\text{mean}}{x.\text{stddev}}$$

$$x = \frac{x - \mu}{\sigma}$$

Train-test split.

create 2 different arrays for x & y
divide to x_{tr} & x_{te} , y_{tr} & y_{te}

Model training.

$x = 0$

$h(x) = \theta_0 + \theta_1 x$ hypothesis function.

$$\text{Error} = \frac{1}{2m} \sum_{i=1}^m (y - y^{(i)})^2$$

Maths

$$\begin{aligned} J(\theta) &= \frac{1}{2m} \sum_{i=1}^m [h(\theta^{(i)}) - y^{(i)}]^2 \\ &= \frac{1}{2m} \sum_{i=1}^m [\theta_0 + \theta_1 x^{(i)} - y^{(i)}]^2 \end{aligned}$$

$$\begin{aligned} \frac{\partial J(\theta)}{\partial \theta_0} &= \frac{1}{m} \sum_{i=1}^m [\theta_0 + \theta_1 x^{(i)} - y^{(i)}] \cdot 1 \\ &= \frac{1}{m} \sum_{i=1}^m [\theta_0 + \theta_1 x^{(i)} - y^{(i)}] \end{aligned}$$

$$\begin{aligned} \frac{\partial J(\theta)}{\partial \theta_1} &= \frac{1}{m} \sum_{i=1}^m [\theta_0 + \theta_1 x^{(i)} - y^{(i)}] x^{(i)} \\ &= \frac{1}{m} \sum_{i=1}^m [h(\theta^{(i)}) - y^{(i)}] x^{(i)} \end{aligned}$$

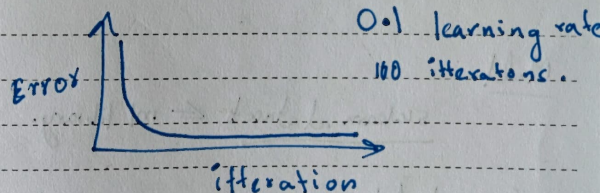
$$\theta_0 = \theta_0 - \eta \cdot \frac{\partial J(\theta)}{\partial \theta_0} \quad \theta_1 = \theta_1 - \eta \cdot \frac{\partial J(\theta)}{\partial \theta_1}$$

code. train

grad.

find error convert math to code.

no. of iteration & learning rate crucial for better outcome.



Predict test with hypothesis and plot.

$$R_2 \text{ score} = 1 - \frac{T_1}{T_2} = 1 - \frac{(y^{(i)} - y_{\text{mean}})^2}{(y^{(i)} - y_{\text{mean}})^2} \quad \text{HQ}$$

$R_2 = 0$ bad $y = y_{\text{mean}}$
 $R_2 = 1$ good $x = \text{data fits on line}$

for multiple features.

$$\text{hypothesis} = \theta^T x$$

$$J = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

training:

$$\theta_0 x_0 + \theta_1 x_1 + \dots + \boxed{\theta_j x_j} + \dots + \theta_n x_n$$

$$\text{gradient: } \frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_\theta(x^i) - y^i) x_j^i$$

grad
update
to
 θ_j

$$\theta_j = \theta_j - \eta \cdot \frac{\partial J(\theta)}{\partial \theta_j}$$

Model

sklearn.datasets $\leftarrow$ ml library.

Generate data

Normalize data. $x = \frac{x - \mu}{\sigma}$

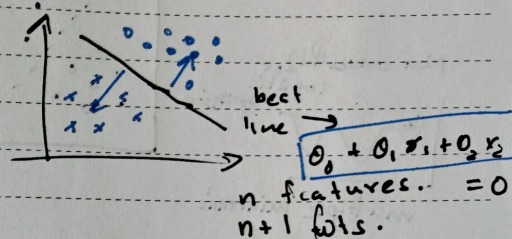
Hypothesis $\text{np.dot}(x, \theta)$ - vector multiplication

Gradient $\frac{\partial J(\theta)}{\partial \theta_j}$ grad = $\frac{\partial J}{\partial \theta_j}$ $\begin{matrix} \text{features} \\ \theta_1 \\ \theta_2 \end{matrix}$

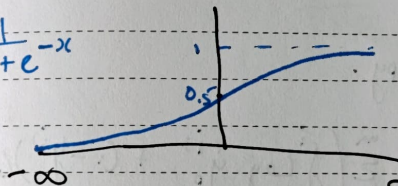
Training.
R2 score.

Logistic Regression

classification based upon supervised learning.
binary classification o/p = 0/1



$$\sigma = \frac{1}{1 + e^{-x}}$$



$$\sigma(d) = \begin{cases} 1 & d \rightarrow \infty \\ 0.5 & d = 0 \\ 0 & d \rightarrow -\infty \end{cases}$$

hypothesis $\hat{y}_p = h_\theta(x) = \sigma(\theta^T x)$

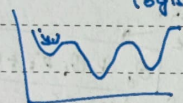
$$= \frac{1}{1 + e^{-\theta^T x}} = \frac{1}{1 + e^{-\sum_{i=1}^n \theta_i x_i}}$$

at line $\Rightarrow d=0$
 $\sigma(0) = 0.5$

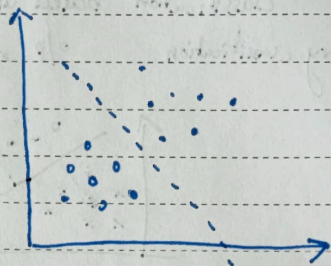
o/p by applying sigmoid function.

Loss Function

Not convex for logistic regression



multiple minima

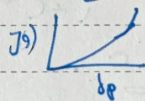
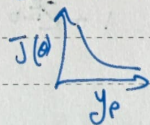


Binary Crossentropy

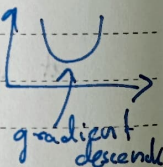
$$J(\theta) = \frac{1}{m} \sum_{i=1}^m (y^i \log \hat{y}^i + (1 - y^i) \log(1 - \hat{y}^i))$$

Measures the discrepancy b/w predictions & true labels.

$$\hat{y}^i = [1, 1, 1, 1, 1] \quad [0, 0, 0, 0, 0]$$



combine



Training

$$\theta_j = \theta_j - \eta \cdot \frac{\partial J(\theta)}{\partial \theta_j}$$

derivative of sigmoid fn.
 $= g(z)(1 - g(z))$

$$\frac{\partial J(\theta)}{\partial \theta_j} = -(\theta^T x^i)(1 - \theta^T x^i) = y^i(1 - y^i)x_j^i$$

$$= -\frac{1}{m} \sum_{i=1}^m [y^i - y^i \theta^T x^i] x_j^i$$

$$\frac{\partial J(\theta)}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m [y^i - h_\theta(x^i)] x_j^i$$

Code

sklearn.datasets make_blobs

Generate, Normalize.

Train test split.

Model

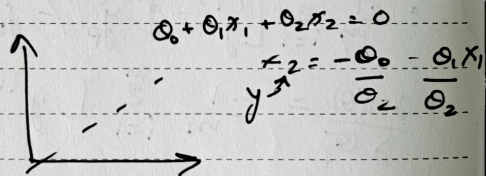
def sigmoid funct. = hypothesis
 $\frac{1}{1 + e^{-z}}$

loss function - binary cross entropy.
error -
loss function.

Model training: init theta.
yp = hypothesis.
error list.
grad

theta = theta - learning rate * grad.

Visualize the line



def prediction.

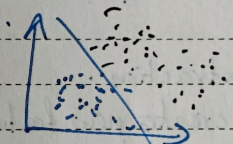
$n \times 1$ $\begin{bmatrix} 0.9 \\ 0.6 \\ 0.3 \\ 0.4 \end{bmatrix}$ $\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$ $\rightarrow$ $\begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$ preds = np.zeros((x.shape[0], 1)).
 $\text{preds}[\text{preds} \geq 0.5] = 1$

sklearn library for logistic regression.

Multi-class classification.
more than 2 classification.

Split the whole into multiple binary classifiers

1 vs 1 | 1 vs Rest.
splitting multi class into 1 binary classification per class.



each classifier gives a prediction. which has.
more prediction value checked

One vs One strategy.

for each class versus every other

$N(N-1)/2$ classifiers for N classes.

each predicts one class label and higher prediction taken.

eg. support vector machines.

Curse of Dimensionality.

→ irrelevant features

→ redundant features

height ✓
in cm in mm

Dimⁿ reducⁿ

Feature Selection.

subset of most relevant features from other features.

Feature Extraction.

: creates new features from existing features.

Filter method.

checks relevance b/o w/p & i/p.

Pearson correlation coeff.

Spearson rank.

do before ML Algo

Wrapper method

during ML Algorithm.

Forward selection.

← Reverse Backward Elimination

check the features with ML and check perf

rank the highest and cross with other and check.

A → Model Perf.

AB
AC

→ AC

ACB
ACD

→ ACD

Embedded Method.

Combines the qualities of filter & wrapper methods

L1 regularisation, Decision trees, etc.

PCA method.

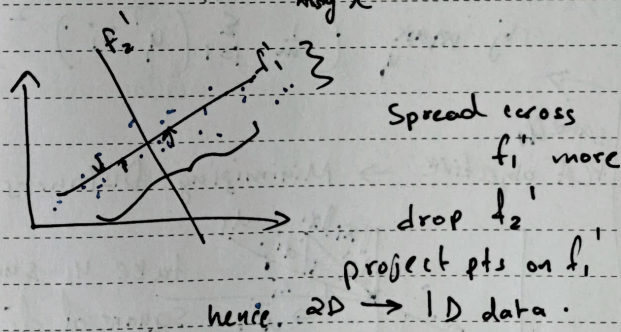
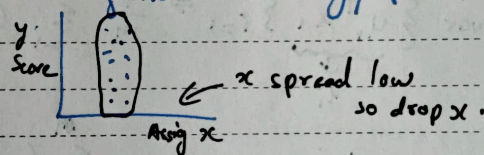
feature extraction

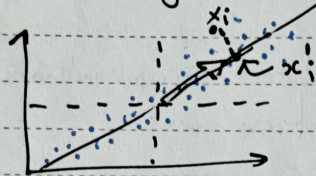
convert n features to k , $k \leq n$

Applications: Data compression, Data visualisation, Speed up Computation.

unsupervised → doesn't look at y data.

preserve the dims having max variability/spread.



Maximizing Variance: u_1 

← 2D

 u_1 : unit vector

$$\|u_1\| = 1$$

$$x_i' = \text{proj}_{u_1}(x_i)$$

$$x_i' = \frac{u_1 \cdot x_i}{\|u_1\|} \Rightarrow 1$$

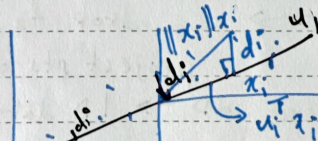
$$1D \Rightarrow x_i' = u_1^T \cdot x_i$$

$$\sum u_1 \cdot x_i$$

find u_1 such that $\text{var}\{\text{proj}_{u_1}(x_i)\}$ is maximum.

$$\text{var} \left\{ \underbrace{u_1^T x_i}_{x_i'} \right\}_{i=1}^m \Rightarrow \frac{1}{m} \sum_{i=1}^m \underbrace{(u_1^T x_i)^2}_{x_i'^2}$$

$$\text{obj max}_y \left(\frac{1}{m} \sum_{i=1}^m (u_1^T x_i)^2 \right)$$

same u_1 PCA objective $\rightarrow$ Minimizing Distancestake u_1 such that
square of d_i sum

it should be minimum

$$\min_{u_1} \sum_{i=1}^m (x_i^T x_i - (u_1^T x_i)^2)$$

Eigen value decomposition.

$$S = \frac{1}{n} X^T X$$

$$\lambda_1 \geq \lambda_2 \geq \lambda_3 \dots$$

eigen values (S) $\lambda_1, \lambda_2, \lambda_3 \dots \lambda_n$ eigen vectors (S) $v_1, v_2, v_3 \dots v_n$ ↑
var.

eqn

$$\lambda_1 \cdot v_1 = S \cdot v_1 \quad \& \text{ this satisfies.}$$

 $\lambda_1 = \text{Eigenvalue}$ $v_1 = \text{eigen vector}$

$$u_1 = v_1$$

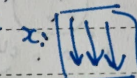
max variance direction.

$$S = \lambda_1 v_1$$

PCA summary.

• column standardise

mean centric



take mean & subtract

$$S = \frac{X^T \cdot X}{m}$$

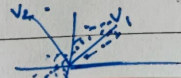
• compute eigen values & vectors of S.

$$u_1 = v_1 \quad u_2 = v_2$$

• select top k eigen vectors.

• Project x_i on selected eigen vectors.

In eigen values & vectors are mapped and spread taken



$$\lambda = \frac{\lambda_1}{\lambda_1 + \lambda_2} = 61\% \quad \lambda_2 = 39\% \quad \text{HO.}$$

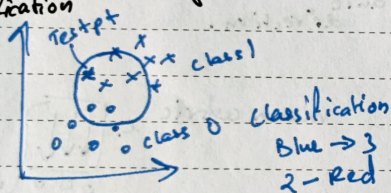
Parametric Models.

hypothesis can be fixed, Parameters no: don't change with data size.

Non-Parametric Methods. decision trees, change parameters
K nearest Neighbours.

K nearest Neighbours KNN

Classification



Pred Blue

$$\frac{3}{5} \times 100 = 60\%$$

1) Measure dist to all m pts

2) Sort all to distances.

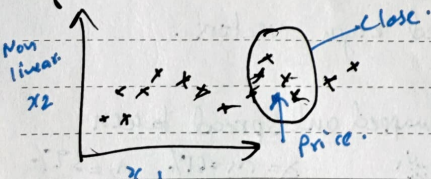
3) K-Nearest points $K=5$

- load training data
- Find similarity or dist of test x & training
- Sort the distance / similarity
- Take mean of nearest k values for regression.

take Avg house prices

$$K=3$$

Regress.



code.

generate data.

def distance

$$np.sqrt(np.sum((p-q)**2))$$

def KNN(x, y, xt, k=5).

$m = x.shape$

dist = []

for i in range(m):

$d = \text{dist}(x[i], xt)$

dist.append((d, y[i]))

dist = sorted(dist)

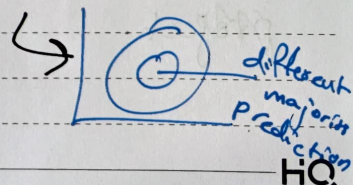
dist = np.array(dist[:k])

labels = dist[-1]

labels [0 2]

another distance method. $\text{city} [1 4]$
Manhattan distance

choosing the right value of k . $idx = 1$



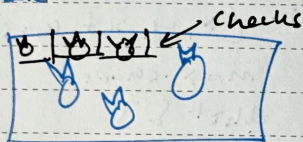
OpenCV

import cv2.

cv2.VideoCapture.

Haarcascade classifiers

in perspective of scale & count.



From opencv download the template.

1.3 30%.

model = cv2.CascadeClassifier("haarcascade_frontalface.xml")
 faces = model.detectMultiScale(1.3, 30%)

for i in faces:

x, y, w, h = f

cv2.rectangle(img, (x, y), (x+w, y+h),
 (255, 0, 255), 2)

plt.imshow

project.

K means clustering

cluster data pts to k groups.

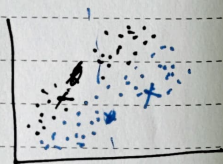
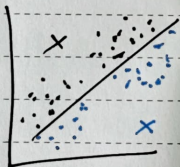
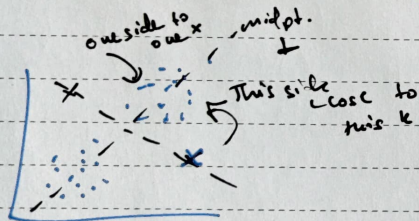
o/p is a label that assign to a class.

creating a centroid for each group. they capture the points closest to them.

Steps

1. Define k centroids. initialize these at random.

2. a) update cluster assignments. assign each to one of k clusters.
b) Move the centroids to be center of their cluster.



Project Dominant colours.

all colours

	R	G	B
Pix[1]	255	0	110
Pix[2]	0	150	200

8 bit int
0-255

find cluster centroids of required colours.

then find dominant colours.

import k means from sklearn.

$k=k$ clusters

model = KMeans(n_clusters= k).

model.fit(x) → image. $x = \text{img.reshape}(-1, 3)$

centroids = model.cluster_centers_

print(centroids)

colors = np.array(centroids, dtype='u int8')
print(colors)

for i in range(colors):
 supp = 0
 mat = np.zeros((100, 100, 3), dtype='uint8')
 mat[:, :, :] = colors[i]
 plt.imshow(mat)
 plt.show()

Bayes Theorem.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

$\uparrow$ Likelihood $\rightarrow$ Prior probab.
 $\downarrow$ Posterior probability $\rightarrow$ marginal/evidence.

A, B $\rightarrow$ any events.

$P(A|B) \rightarrow$ B already happened, what's Prob of A happening with it
 $= \frac{P(A \cap B)}{P(B)}$
 $\uparrow$ intersection commutative.

$P(B|A) = \frac{P(B \cap A)}{P(A)}$
 equate.

$$P(A|B) \cdot P(B) = P(B|A) \cdot P(A)$$

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Qn.

A $\rightarrow$ rain on Saturday
 B $\rightarrow$ rain on Sunday

$$P(A) = 0.25$$

$$P(B|A) = 0.5$$

$$P(B|A^c) = 0.25$$

$\frac{P(B)}{P(B)} \rightarrow$ rain on Sunday
 rain on Saturday

$\rightarrow$ B
 not rain on Saturday

$$P(B) = P(A) \cdot P(B|A) + P(A^c) \cdot P(B|A^c)$$

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)} = 0.5125$$

$$P(B) = 0.4 = 40\%$$

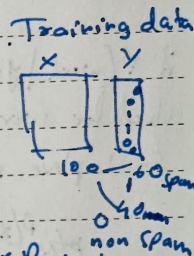
Naive Bayes Algorithm.

Spam prediction.

$$P(Y=0|x) = \frac{P(x|Y=0) \cdot P(Y=0)}{P(x)}$$

x email content

$$P(Y=1|x) = \frac{P(x|Y=1) \cdot P(Y=1)}{P(x)}$$



words in check if the test mail has similar words in class which is non spam.

$P(Y=0) = \frac{\text{count}(0)}{\text{total}} = 0.6$
 $P(Y=1) = \frac{\text{count}(1)}{\text{total}} = 0.4$

$$P(Y=0|x) \propto P(x|Y=0) \cdot P(Y=0)$$

$$P(Y=1|x) \propto P(x|Y=1) \cdot P(Y=1)$$

words similar count in spam check.

we ignore $P(x) \cdot \text{const.}$

$$P(x) = P(x|Y=0) \cdot P(Y=0) + P(x|Y=1) \cdot P(Y=1)$$

Law of total probability.

$$x = \langle x_1, x_2, \dots, x_n \rangle$$

words uniformly free/data

$$P(x, Y=1) = P(x_1|Y=1, x_2, \dots, x_n) \cdot P(x_2|Y=1, x_1, x_3, \dots, x_n) \dots P(x_n|Y=1, x_1, \dots, x_{n-1})$$

but computation takes time

So assumes Naive Bayes

Posterior prob. $\rightarrow$ independent features, independence conditionally

$$P(Y=1|x) = \prod_{i=1}^n \frac{P(x_i|Y=1)}{P(x)}$$

$$P(Y=0|x) = \prod_{i=1}^n \frac{P(x_i|Y=0) \cdot P(Y=0)}{P(x)}$$

Multinomial Naive Bayes.

If $P(\text{word} | y=1)$ is not there, then conditional prob = 0.

Used for word classification.

chinese chinese

So Laplace smoothing

$$P(x_i | y=1) = \frac{P(\text{count} | y=1) + \alpha}{P(\text{all words in } y=1) + \alpha(V)}$$

a random const
→ unique words.

Bernoulli Naive Bayes.

Doesn't talk about frequency of word.

concerns only on if present or not.

→ conditional

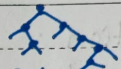
$$P(x_i | y_i = c) = \frac{\text{count}(d_i \text{ contains } x_i, y_i = c) + \alpha}{\text{count}(y_i = c) + 2\alpha}$$

$$P(Y=1 | X) = \prod_{i=1}^{|V|} P(x_i | y=1)^{a_i} (1 - P(x_i | y=\text{spam}))^{1-a_i}$$

Posterior.

Gaussian Naive Bayes.

Decision trees

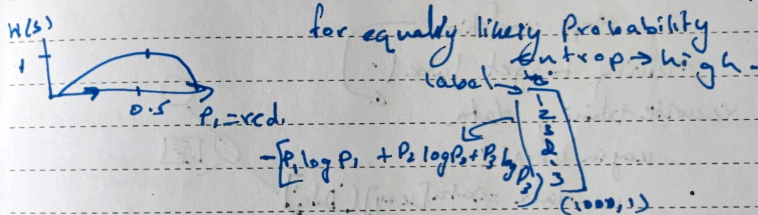


supervised models, used for Both classification/regression
nested if-else, Highly interpretable models.

Entropy is measure of randomness of a system.
uncertainty ↑ entropy ↑

$$H(X) = - \sum_{i=1}^n P_i \log_2 P_i$$

$Y \rightarrow$ output label
 $P \rightarrow$ Probability. $-1 + (-1) + 1$



Information Gain

reduction in entropy is Information gain.

if any feature reduces the entropy → its good.

→ How many split

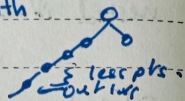
$$\text{Gain}(Y, A) = H_D(Y) - \sum_{i=1}^K \frac{|D_i|}{|D|} H_{D_i}(Y_i)$$

split by A feature without split Entropy entropy of split feature

↓ weight factor. Total data pt.

Stopping Condition:

1. Pure Node.
2. Can't grow tree because of lack of pts
3. It already reach a max depth



code from scratch

iterate over all features

which feature has max info gain
split accord. to that feature

stopping condition

assign target label

recursive training data

key in keys

newdata = data[key][data]

Decision tree (depth = self.depth + 1)

return

model = Decision Tree()

model.train (dp)

Handling Numeric features by Decision Tree

Bias Variance Trade off: depth

overfitting previn

Hyper parameter tuning

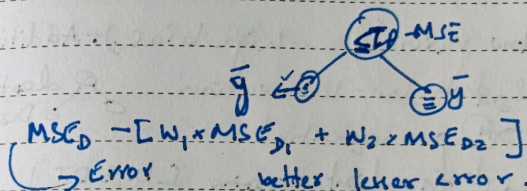
Pruning of trees cutting tree from bottom up
Are it i add this i will fit good better results to up down

Decision tree for Regression.

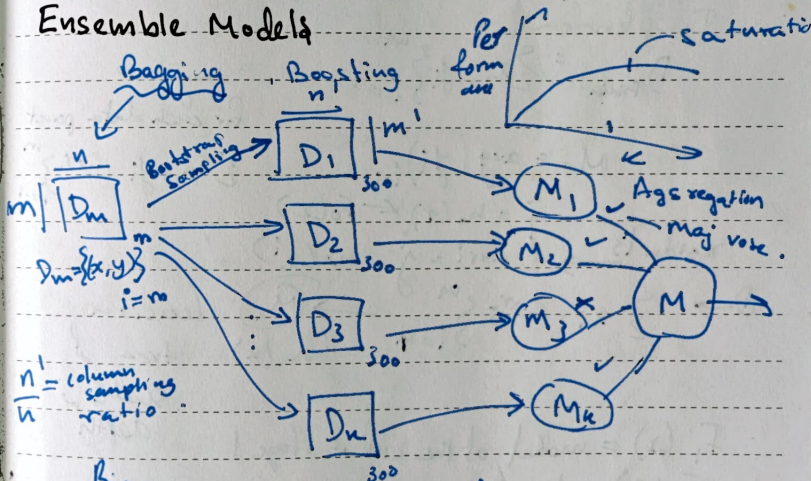
classification $\rightarrow$ In.

Regression $\rightarrow$ MSE.

target value = mean of 'y'



Ensemble Models



$\frac{n'}{n}$ = column sampling ratio

Binary

Classification

Majority Vote

for regression

mean, median

HC

reduce variance $\rightarrow$ not affected on bias.

random forest:

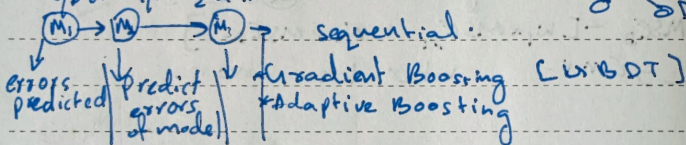
RF = D.T + R.S + C.S + Aggregation
(large depth) $\rightarrow$ low sampling (mean/median)

Boosting

Models (low variance - high bias) + Additively $\rightarrow$ low variance

o/p of 1st/2nd depth low-low variance

depth-1
Decision stump



Implement

$$D_{\text{train}} = \{x_i, y_i\}_{i=1}^m$$

avg. $\rightarrow$
for each data point
 $\{x_i, y_i, \text{err}_i\}_{i=1}^m$

$$M_0 = \text{avg}(y_i)_{i=1}^m$$

$$\hat{y} = M_0(x) \leftarrow f_0(x)$$

residuals $\rightarrow$ $\text{err}_i = y_i - M_0(x_i)$

$$\textcircled{1} \rightarrow \{x_i, \text{err}_i\}_{i=1}^m \rightarrow \textcircled{M_1} \text{ Decision tree of small depth}$$

$F_1(x)$ = model at the end of stage 1

$$F_1(x) = M_0(x) + \alpha_1 h_1(x)$$

$\downarrow$
Prediction of model 1 (Avg)

$\downarrow$
Error pred from model 1

$$\textcircled{2} \text{err}_i = \{y_i - f_1(x_i)\}_{i=1}^m$$

$$\frac{\sum x_i \cdot \text{err}_i}{\sum y_i}$$

$\leftarrow M_2$
($h_2(x)$)

At the end of stage, each model helps next model.

$$F_k(x) = M_0(x) + \sum_{i=1}^k \alpha_i h_i(x)$$

$h_k(x) \rightarrow$ trained on $\{x_i, \text{err}_i\}$

loss Minimization $\rightarrow J_k = F_{k,1}(x_i)$

$$L(y, F_k(x_i)) = (y_i - \hat{y}_i)^2 \quad \therefore \hat{y}_i = F_k(x_i)$$

$$-\frac{\partial L}{\partial F_k(x_i)} = y_i - F_k(x_i) \text{ x residual}$$

$\downarrow$ if neg residual derivative $\rightarrow$ pseudo residuals.

Gradient Boosting Algorithm //

Overfitting

$$M \rightarrow \# \text{ of models } f_2(x)$$

$$M \downarrow = 2 \text{ [Underfit]}$$

$$M \uparrow \uparrow = 2000 \text{ [Overfit]}$$

$$F_m(x) = F_{m-1}(x) + \eta \cdot \gamma_m h_m(x)$$

learning rate $0 < \eta \leq 1$

Grid Search

$$m=100 \quad \eta=0.1$$

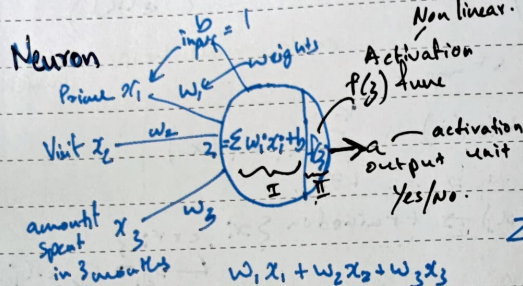
cross-validation

$\eta=1$ overfitting $\uparrow$

$\eta=0.0001$ Overfitting $\downarrow$, underfitting $\uparrow$

XG Boost

Introduction to Deep Learning -



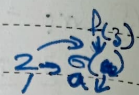
$$z = \sum_{i=1}^n w_i x_i + w_0$$

$$a = g(z)$$

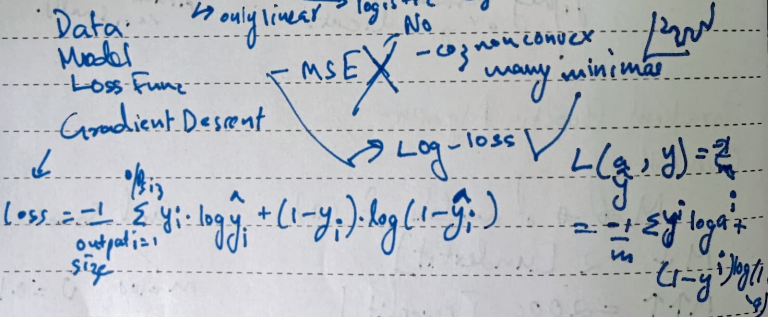
$$z = \sum_{i=1}^n w_i x_i + b$$

$$\theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n + \theta_0$$

Sigmoid
tanh
ReLU

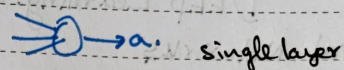


How does a Perceptron Learn



Neural Network

1. Perceptron



2. Layer Neural Network

- MLP - Multi layer Perceptron
 2 layers Min.

